

# BizTrack R Analytics

From knowing zero R → explaining every line like you wrote it

A ground-up walkthrough of the `r/` prototype: the language, the statistics, and every file — the synthetic data generator, the SPC control charts (Feature 7), the discrete-event simulation (Feature 6), the API, and the Shiny dashboard.

Read it front to back on the commute. By the end you'll be able to sit in front of the code and narrate *what it does, why it's built that way, and what you'd say if a panelist pokes it*.

# Contents

## Part 0 — The 60-second picture

What the whole thing does

## Part 1 — R from absolute zero

What R even is

Variables & printing

Everything is a vector

Data types & dates

Functions

Randomness & set.seed

if / ifelse / loops / apply

The pipe |>

Packages

## Part 2 — The tidyverse dialect

Tibbles

The dplyr verbs

Joins & binds

case\_when, expand\_grid, glue, dates

## Part 3 — The stats & OR ideas

Random distributions

Why lognormal (the mean trick)

SPC / control charts

DES / simmer

RA 11032 & working days

## Part 4 — The codebase, file by file

config.R

R/generate.R

R/analytics.R

R/spc.R

R/des.R

plumber.R + run\_api.R

R/db.R

run\_all.R

app.R (Shiny)

install.R

## Part 5 — Explain it like you made it

Design decisions & likely questions

One-page cheat sheet

## Part 0 – The 60-second picture

---

Before any code: what is this thing, in one breath?

Your paper promises that BizTrack (a business-permit tracking system for a local government) can be enriched with **R** — a language built for statistics — to do two "headline" analytics features. This prototype proves both work, end to end, offline, on a laptop, reproducibly. It does that in three moves:

1. **Manufacture believable data.** There's no real permit database yet, so `R/generate.R` invents 36 months of realistic permit history — businesses, applications, department reviews, inspections, issued permits — and saves them as CSV files. *On purpose*, it also sneaks a "slowdown" into one department's last 6 weeks.
2. **Feature 7 — Statistical Process Control ( `R/spc.R` ).** Draw a *control chart* of each department's weekly speed and automatically flag weeks that are abnormally slow. The win: it **catches the slowdown we deliberately injected** — proving the technique detects real problems, not noise.
3. **Feature 6 — Discrete-Event Simulation ( `R/des.R` ).** Build a little simulated model of the permit office and ask "*what if we add 2 fire inspectors?*" It answers with numbers: on-time compliance goes up by *X* points. That's an operations-research staffing tool.

Everything else is scaffolding around those three: `config.R` (settings), `analytics.R` (shared calculations the paper cites), `run_all.R` (the one command that runs it all), `plumber.R` (a tiny web API so the main app could fetch the results), `app.R` (the Shiny dashboard you present), and `db.R` (placeholder readers for when a real database is wired up later).

**The story you're really telling.** "We inject a known fault, then show our statistics find it; and we simulate the office so staffing decisions can be tested before spending money." Detection + prediction. Keep that sentence in your pocket.

## Part 1 – R from absolute zero

---

No prior R assumed. Read this part and the code stops looking like hieroglyphics.

### What R even is

R is a programming language whose native talent is **data and statistics**. Where Python is a general tool that added data libraries, R was *born* for tables, distributions, and charts — so a lot of things that are fiddly elsewhere are one word here ( `mean(x)` , `median(x)` , fit a distribution, draw a chart).

You run our project with **Rscript**, the command-line runner:

```
# from inside the r/ folder
Rscript run_all.R      # runs the whole pipeline top to bottom
```

`Rscript somefile.R` just means "execute this R file line by line and stop." That's literally all `run_all.R` is — a script that calls the other pieces in order.

### Variables & printing

R's assignment operator is `<-` (a left arrow). `x <- 5` reads "x gets 5". You *can* use `=` too, but R culture strongly prefers `<-` , and our whole codebase uses it.

```
SEED <- 1103          # a number
MODE <- "synthetic"   # text (a "string"); quotes required
MONTHS <- 36

print(SEED)           # show it → [1] 1103
SEED                  # typed alone in an interactive session, also prints it
```

The `[1]` R prints in front of results is just "this line starts at element #1" — a hint that *everything in R is a vector* (next section). Names in `ALL_CAPS` like `SEED` are a convention for constants; ordinary variables are usually lowercase.

### Everything is a vector

This is *the* mental unlock for R. In most languages a number is a single value. In R, a single number is secretly a vector of *length 1*. You build longer vectors with `c()` ("combine"):

```
reviewers <- c(3, 2, 2)      # a numeric vector, length 3
depts     <- c("BPLO", "CH0", "BFP") # a character vector
```

Because vectors are the default, math is **vectorised** — it happens to every element at once, no loop needed:

```
reviewers * 2          # → 6 4 4   (multiplies each element)
reviewers + c(1,1,1)   # → 4 3 3   (element-by-element)
mean(reviewers)        # → 2.33   (one number summarising the vector)
```

You'll see this everywhere in our generator: it creates, say, 3,800 review durations in *one* vectorised expression instead of a 3,800-iteration loop. That's the R way.

**Indexing** uses square brackets, and R counts from **1** (not 0):

```
depts[1]              # → "BPL0" (the FIRST element)
depts[c(1,3)]         # → "BPL0" "BFP"
reviewers[reviewers > 2] # → 3   (keep elements where the test is TRUE)
```

That last form — indexing with a TRUE/FALSE vector — is called **logical indexing** and it's how the generator does things like "keep only applications submitted before the anchor date."

## Data types & dates

| Type               | Looks like          | Where it shows up here                      |
|--------------------|---------------------|---------------------------------------------|
| numeric            | 2.5 , 90            | counts, durations, rates                    |
| character (string) | "CH0"               | department codes, statuses                  |
| logical            | TRUE / FALSE        | on_time flags, filters                      |
| Date               | 2026-06-30          | calendar days (no time)                     |
| POSIXct            | 2026-06-30 14:03:00 | timestamps (date <i>and</i> time)           |
| NA                 | NA                  | "missing / unknown" (e.g. not-yet-approved) |

Dates matter a lot in this project because the whole thing is about *how long* permits take. Two facts to remember:

- **Date** = a calendar day. **POSIXct** = an exact instant. The generator submits an application at a POSIXct timestamp (e.g. 2:03pm) and later measures the gap to approval.
- Subtracting two timestamps gives a *duration*. The code always pins the unit explicitly:  
`as.numeric(approved_at - submitted_at, units = "days")` — "how many days elapsed, as a plain number." Without that you can get hours or weeks depending on R's mood, so we're explicit.

**NA** is R's "missing value." It's contagious on purpose: `NA + 5` is `NA`, because unknown-plus-five is still unknown. That's why you'll see guards like `!is.na(approved_at)` — "only keep rows where approval date is *not* missing."

## Functions

A function is a named recipe: give it inputs (arguments), get back a value. You call one with `name(args)`. You define one with `function(...)`:

```
# DEFINE: a recipe that turns a target average into lognormal draws
rlnorm_mean <- function(n, mean_target, sdlog = 0.4) {
  meanlog <- log(mean_target) - (sdlog^2) / 2
  rlnorm(n, meanlog = meanlog, sdlog = sdlog) # last line = the return value
}

# CALL it
rlnorm_mean(100, mean_target = 2.5) # 100 draws averaging ~2.5
```

Three things to notice, because they recur constantly in our code:

- **The last expression is the return value.** No `return` keyword needed (though it exists). Whatever the function computes last is what comes back.
- **Default arguments:** `sdlog = 0.4` means if the caller doesn't supply `sdlog`, it's 0.4. So `rlnorm_mean(100, 2.5)` works.
- **Named arguments:** `meanlog = meanlog` — you can pass arguments by name, in any order. This makes calls self-documenting.

## Randomness & `set.seed()`

This project leans hard on random numbers (to fake data and to simulate). But "random" would make results different every run — bad for a paper you must *reproduce*. R's fix is the **seed**:

```
set.seed(1103)
runif(3) # → 0.207 0.861 0.351 (three "random" numbers 0–1)

set.seed(1103)
runif(3) # → 0.207 0.861 0.351 (IDENTICAL – same seed, same stream)
```

A seed makes randomness **deterministic**: same seed  $\Rightarrow$  same numbers, forever, on any machine. That's why `config.R` has `SEED <- 1103` and the generator starts with `set.seed(seed)`. It's the single most important line for the "reproducible" claim in your paper. (The simulation goes further: `set.seed(SEED + r)` per replication `r`, so each of the 30 repeats is random-but-reproducible.)

## if / ifelse / loops / the apply family

R has two different "if"s and it matters which you use:

```
# if() – makes ONE decision (scalar). Used for control flow.
if (MODE == "synthetic") {
  message("using fake data")
} else {
  message("using the database")
}

# ifelse() – decides for EVERY element of a vector at once (vectorised)
complexity <- ifelse(application_type == "new", "complex", "simple")
# for 3,800 applications, tags each one "complex" or "simple" in one shot
```

**Loops exist** but R programmers avoid them for data work, preferring the **apply family** — functions that run a small function over each element and collect the results:

### **lapply(x, f)**

run `f` on each item of `x`, return a **list**. Used in `spc.R` to compute limits for each department.

### **vapply(x, f, template)**

like `lapply` but you promise the output type (e.g. `character(1)` = "each result is one string").  
Safer/faster. The generator uses it to pick each application's type.

### **mapply(f, a, b)**

walk two vectors in lockstep (month-start *and* that month's volume).

Mentally, `lapply(depts, compute_limits)` = "for each department, compute its control limits, hand me back the collection." It's a loop with the bookkeeping hidden.

## **The pipe: `|>`**

This symbol is *all over* our code and once it clicks, everything reads like English. The pipe takes whatever is on its left and feeds it as the first argument to the function on its right. These two are identical:

```
# without the pipe – read inside-out, painful
arrange(summarise(group_by(assignments, department_code), n = n()), department_code)

# with the pipe – read top-to-bottom, like a sentence
assignments |>
  group_by(department_code) |>
  summarise(n = n()) |>
  arrange(department_code)
```

Read `|>` as the word "**then**": "take assignments, *then* group by department, *then* summarise, *then* arrange." Every multi-step data transformation in this project is a pipe chain. When you present the code, literally narrate the pipe out loud and you're explaining it.

**Two pipes exist.** You may see `%>%` (from the `magrittr`/tidyverse package) in tutorials online; our code uses R's newer *built-in* pipe `|>`. Same idea. If asked: "we use the native base-R pipe, no extra dependency."

## Packages

R ships with a core, and everything else is **packages** you install once and load per script with `library()`. `install.R` installs ours; each file opens with the ones it needs:

```
library(dplyr)      # data verbs (filter, mutate, group_by, ...)
library(ggplot2)    # charts
library(simmer)     # discrete-event simulation engine
library(qcc)        # quality-control / control charts
```

Here's the map of the ones that matter, so a package name never mystifies you:

| Package                                                   | Its one job here                                           |
|-----------------------------------------------------------|------------------------------------------------------------|
| <code>dplyr</code> , <code>tidyr</code>                   | reshape and summarise tables (the verbs)                   |
| <code>readr</code>                                        | read/write CSV files                                       |
| <code>lubridate</code>                                    | date/time arithmetic (months, weeks, floor to week)        |
| <code>tibble</code>                                       | the modern data-table type                                 |
| <code>ggplot2</code>                                      | build the charts                                           |
| <code>glue</code>                                         | slot variables into text: <code>glue("seed {SEED}")</code> |
| <code>qcc</code>                                          | <b>Feature 7:</b> control charts (xbar, EWMA)              |
| <code>simmer</code>                                       | <b>Feature 6:</b> the simulation engine                    |
| <code>fitdistrplus</code>                                 | fit a lognormal to observed durations                      |
| <code>plumber</code>                                      | turn R functions into a web API                            |
| <code>shiny</code> , <code>bslib</code> , <code>DT</code> | the interactive dashboard                                  |

**Why the `dplyr::select` spelling shows up.** Two packages both define a `select`. Writing `dplyr::select` (package-name, then `::`, then function) says "the one from `dplyr`, no ambiguity." If someone asks why some calls are qualified like that, that's the honest answer — we hit a name clash and pinned it.



## Part 2 – The tidyverse dialect

Our code isn't "plain R" — it's written in the **tidyverse** style: tables in, tables out, connected by pipes. Learn ~10 verbs and you can read all of it.

### Tibbles – the table

A **tibble** is R's modern spreadsheet: named columns, each an equal-length vector, rows are records. It's what every CSV becomes when loaded and what every function returns.

```
DEPARTMENTS <- tibble(  
  code      = c("BPLO", "CH0", "BFP"),  
  name      = c("Business Permits...", "City Health...", "Bureau of Fire..."),  
  reviewers = c(3, 2, 2),  
  mean_days = c(2.0, 2.5, 3.0)  
)
```

That's a 3-row, 4-column table. Grab a column with `$ : DEPARTMENTS$reviewers → 3 2 2`. (A tibble is the friendlier cousin of R's older `data.frame`; treat them as the same idea.)

### The dplyr verbs – the whole vocabulary

Nearly every data operation in the project is one of these seven verbs, chained by pipes. This table is the Rosetta stone for reading the code:

| Verb                     | Plain English                      | Tiny example                                                 |
|--------------------------|------------------------------------|--------------------------------------------------------------|
| <code>filter()</code>    | keep rows matching a condition     | <code>filter(status == "approved")</code>                    |
| <code>mutate()</code>    | add/replace a column               | <code>mutate(on_time = approved_at &lt;= deadline_at)</code> |
| <code>select()</code>    | keep/reorder columns               | <code>select(id, department_code)</code>                     |
| <code>arrange()</code>   | sort rows                          | <code>arrange(week_start)</code>                             |
| <code>group_by()</code>  | form groups for the next summarise | <code>group_by(department_code)</code>                       |
| <code>summarise()</code> | collapse each group to one row     | <code>summarise(n = n(), avg = mean(x))</code>               |
| <code>n()</code>         | "how many rows in this group"      | used inside summarise                                        |

The killer combo is **group\_by + summarise** — "split into groups, compute a number per group." Here is the *real* workload calculation from `analytics.R`, fully annotated:

```

assignments |>
  filter(!is.na(completed_at)) |> # drop unfinished reviews
  mutate(turnaround = # add a "days taken" column
    as.numeric(completed_at - assigned_at, units="days")) |>
  group_by(department_code) |> # one bucket per department
  summarise( # collapse each bucket → one row
    n_assignments = n(), # how many reviews
    mean_turnaround_days = mean(turnaround) # their average speed
  ) |>
  mutate(workload_share = n_assignments / total) |> # each dept's % of all work
  arrange(desc(n_assignments)) # busiest first

```

Narrated: "Take assignments, drop the unfinished ones, compute each review's turnaround in days, group by department, and for each department report the count and the average turnaround; then add each department's share of the total and sort busiest-first." You just explained a function.

## Joins & binds – gluing tables together

Real data lives in separate tables that share keys (an application has an `id` ; an assignment references that `id`). **Joins** stitch them:

```

assignments |>
  left_join(applications, by = c("application_id" = "id"))

```

`left_join` = "keep every row on the left, and glue on matching columns from the right where the keys line up." The `by = c("application_id" = "id")` says "match the left's `application_id` to the right's `id`." This is how the generator attaches each review's submission time, and how the SPC chart attaches each week's in/out-of-control status.

`bind_rows()` is the vertical version — stack tables on top of each other (same columns, more rows). The generator uses it to combine "every application gets a BPLO review" with "complex ones also get CHO + BFP reviews" into one assignments table.

## A few more you'll meet

### `case_when()`

a multi-branch `ifelse`. The seasonality function: `case_when(month==1 ~ 2.5, month==2 ~ 1.6, month==12 ~ 0.7, TRUE ~ 1.0)` = "January → ×2.5, February → ×1.6, December → ×0.7, everything else → ×1."

### `expand_grid()`

every combination of the inputs. `expand_grid(application_id = ids, permit_type = c("BUSINESS", "SANITARY", "FSIC"))` gives each approved application all three permit rows.

## glue()

string templating. `glue("seed {SEED}, {MONTHS} months")` drops the variables' values into the text. It's how every status message and chart title is built.

## lubridate: floor\_date / %m-% / month()

date math. `floor_date(d, "week")` = "snap this date back to its Monday" (used to bucket reviews into weeks). `anchor %m-% months(35)` = "35 months before the anchor." `month(d)` = pull the month number.

**You now have the reading key.** Every file from here on is: load some tables, pipe them through these verbs, and either save a CSV or draw a chart. The rest of this guide is about the *statistical meaning* and the *specific decisions* — the language itself you've just met.

## Part 3 – The statistics & OR ideas

The code is easy once you know *what it's modelling*. Five ideas cover it.

### Random distributions (the `r***` functions)

R generates random numbers from named **distributions** — different "shapes" of randomness. The generator picks the shape that matches reality for each thing it fakes:

| Call                                 | Shape                                             | Used for                                                      |
|--------------------------------------|---------------------------------------------------|---------------------------------------------------------------|
| <code>runif(n, a, b)</code>          | flat between a and b — every value equally likely | small delays ("0.25 to 1.25 days to pick up a file")          |
| <code>rnorm(n, mean, sd)</code>      | bell curve around a mean                          | ±10% month-to-month volume wobble                             |
| <code>rexp(n, rate)</code>           | gaps between random arrivals                      | when the <i>next</i> application shows up (in the simulation) |
| <code>rlnorm(n, ...)</code>          | right-skewed: mostly small, occasional large      | review durations                                              |
| <code>rgamma(n, shape, scale)</code> | also right-skewed, positive-only                  | inspection durations                                          |
| <code>sample(x, n, prob=)</code>     | pick from a set with given odds                   | application type, pass/fail result, barangay                  |

Every `r`-prefixed function means "random draws from this distribution." You don't need the math — just "this line invents N realistic values of that shape."

### Why lognormal – and the mean trick

Think about how long a review *actually* takes. It can't be negative. Most take a normal-ish time, but a few drag on much longer. That right-skewed, positive-only shape is a **lognormal** distribution — the standard choice for service/processing times. That's why reviews use it.

The catch: a lognormal isn't parameterised by its average. You give it `meanlog` and `sdlog` (the mean/spread of the *logarithm*), and its actual average comes out as `exp(meanlog + sdlog2/2)`. But we want to say "CHO averages 2.5 days." So the code inverts the formula — the one clever line in `generate.R`:

```
meanlog <- log(mean_target) - (sdlog^2) / 2
rlnorm(n, meanlog = meanlog, sdlog = sdlog)
```

Translated: "I want the average to be `mean_target` ; solve for the `meanlog` that delivers exactly that, then draw." So `mean_days = 2.5` in `config.R` genuinely produces reviews averaging 2.5 days. **That's the**

**whole point of the `rlnorm_mean()` helper.** If a panelist asks "how do you control the average of a skewed distribution?" — this line is your answer.

## SPC – Statistical Process Control (Feature 7)

SPC is a factory-floor idea from the 1920s: watch a repeating process, and tell apart *normal variation* ("some weeks are a bit slower, whatever") from a *real shift* ("something broke"). The tool is a **control chart**.

How it's built:

1. Measure the process each week — here, a department's **average review turnaround** that week.
2. From a *calm baseline period*, compute the **centre line** (the normal average) and the **control limits**: typically centre  $\pm 3$  standard deviations. The **UCL** (upper) and **LCL** (lower) are the "if you cross this, it's not just noise" fences.
3. Plot every week. Points inside the fences = **in control** (normal). Points outside = **out of control** (investigate).

The specific chart we use is the **X-bar individuals chart** ( `type="xbar.one"` in the `qcc` package) — "one measurement per period." We *also* run an **EWMA** (exponentially-weighted moving average) chart, which is more sensitive to a slow, sustained *drift* that a plain chart might let slide. A week is flagged if *either* rule trips.

**The one subtle-but-smart move: the calibration window.** We compute the limits using only the **first ~24 weeks** — the clean period *before* the injected slowdown. Why? If you let the slow weeks help set the "normal" band, they'd inflate the band and the chart might not flag them — the fault would hide itself. Calibrating on the clean history keeps the fences honest. This is the single most defensible line in the SPC file; know it cold.

**Why this proves anything:** the generator deliberately made CHO  $1.8\times$  slower for the last 6 weeks. If SPC is real, those 6 weeks should punch through the UCL — and they do (6 consecutive out-of-control weeks), while BPLO and BFP stay calm. Injected fault → detected fault. That's the experiment.

## DES – Discrete-Event Simulation (Feature 6)

DES models a system as **entities** (here: permit applications) flowing through **resources** (staff) that can serve only so many at once. The clock jumps from event to event (arrival, start, finish) — hence "discrete-event." It's the standard way to answer "*if I add staff, what happens to wait times?*" without touching the real office. We use the `simmer` package.

Four concepts, and the exact words the code uses:

### resource

a limited server with a capacity. `add_resource("BFP", 2)` = "two fire reviewers." Adding staff in a scenario = raising this number.

### trajectory

the *route* an entity takes — the recipe of steps. Built with `seize` (grab a resource), `timeout` (spend service time), `release` (let it go).

## generator

the arrival tap. `add_generator("complex", traj, function() rexp(1, lam))` = "new complex applications arrive with exponential gaps at rate lam."

## clone + synchronize

how **parallel** work is modelled. A new application's BPLO, CHO, and BFP reviews happen *at the same time*, so we `clone(3, ...)` into three parallel branches and `synchronize(wait = TRUE)` to wait until all three finish before moving on.

Two derived numbers you'll report:

- **Utilisation** = fraction of time a resource is busy (`server/capacity`, averaged over time). 0.9 = "busy 90% of the time" = a bottleneck. The `.tw_mean` helper computes these as *time-weighted* averages (a state that held for 5 days counts 5× a state that held 1 day).
- **On-time compliance** = share of simulated applications that finished within the RA 11032 deadline. This is the headline the scenarios move.

**The experiment:** run a `baseline` (current staffing), then `plus2_bfp_inspectors` and `plus1_cho_reviewer`, each 30 times, and compare on-time compliance. Both add-staff scenarios beat baseline — quantifying the value of hiring before anyone hires.

## RA 11032 & the working-day conversion

RA 11032 (the Ease of Doing Business Act) sets legal deadlines in **working days**: 3 for simple permits, 7 for complex, 20 for highly technical. But the simulation clock and the fitted durations run in **calendar days** (elapsed time, weekends included). Comparing 7 working days against a calendar flow-time would be apples-to-oranges. The fix, in `des.R`:

```
wd_to_cal <- 7 / 5 # a 5-day work week spans 7 calendar days
deadline <- deadline_working_days * wd_to_cal
```

Multiplying the working-day deadline by 7/5 converts it to its calendar-day equivalent, so the comparison is fair. Small line, but it's the difference between a compliance number that means something and one that's silently wrong. (The generator, separately, has a real `add_working_days()` function that steps day by day skipping weekends, for the exact deadline timestamps.)

## Part 4 – The codebase, file by file

---

Now the payoff. With Parts 1–3 in hand, each file is a short read. For every file: what it's for, how it works, and the lines worth pointing at.

### The folder at a glance

```
r/
├─ config.R      # all the settings/constants (the control panel)
├─ install.R     # installs the R packages (run once)
├─ run_all.R     # THE one command – runs the whole pipeline
├─ R/
│   ├─ generate.R # invents the synthetic permit history → 5 CSVs
│   ├─ analytics.R # the shared calculations the paper cites
│   ├─ spc.R      # Feature 7 – control charts
│   ├─ des.R      # Feature 6 – simulation
│   └─ db.R       # future: read from the real database (stubs)
├─ plumber.R     # tiny web API serving the results as JSON
├─ run_api.R     # starts that API
├─ app.R         # the Shiny dashboard (what you present)
├─ run_app.R     # starts the dashboard
├─ data/         # generated CSVs (created at runtime)
└─ outputs/      # charts, result CSVs, summary.md
```

**Flow of control:** `run_all.R` loads `config.R`, then *sources* (loads the functions from) each `R/*.R` file, then calls them in order: `generate` → `analytics` → `SPC` → `DES` → write summary. The dashboard and API are launched separately and just *read* what `run_all.R` produced.

### `config.R` – the control panel

No logic here, just named constants everything else reads. Centralising them is what makes the run reproducible and tunable from one place.



| Constant                                     | Meaning                                                                                              |
|----------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>MODE &lt;- "synthetic"</code>          | use fake data (vs a future "postgres" mode)                                                          |
| <code>SEED &lt;- 1103</code>                 | the reproducibility seed — the whole run hangs off this                                              |
| <code>MONTHS &lt;- 36</code>                 | generate 3 years of history                                                                          |
| <code>ANCHOR_DATE &lt;- 2026-06-30</code>    | the fixed "today" of the dataset (hard-coded, not the real today, so results don't drift day to day) |
| <code>ANOMALY_WEEKS &lt;- 6</code>           | the injected CHO slowdown covers the last 6 weeks                                                    |
| <code>DEPARTMENTS</code>                     | a tibble: each dept's reviewer headcount & target mean review days                                   |
| <code>DEADLINES</code>                       | RA 11032 working-day limits: simple 3, complex 7, highly-technical 20                                |
| <code>BARANGAYS / WEIGHTS</code>             | Malabon's 21 barangays, weighted so a few dominate (realistic)                                       |
| <code>DES_REPS &lt;- 30</code>               | run each simulation scenario 30 times and average (stability vs runtime)                             |
| <code>DES_ARRIVALS_PER_DAY &lt;- 0.85</code> | the tuned caseload the office is simulated under                                                     |

**Be ready to defend two "magic numbers."** `ANCHOR_DATE` is fixed (not `Sys.Date()`) *deliberately* — so the 36-month window and the anomaly window are identical on every machine, every day.

`DES_ARRIVALS_PER_DAY` is a calibrated *representative* caseload, not raw peak volume; the comment in the file explains that raw seasonal peaks would saturate a 2–3 person office (which is exactly the capacity question the scenarios probe), so we simulate a stable rate and flag seasonal arrivals as future work. That's an honest modelling choice, and it's documented in the code.

## R/generate.R – the synthetic history generator

The biggest and most important file: it fabricates the whole dataset the other features analyse. One entry point, `generate_history(months, seed)`, which `set.seed()`s and then builds five tables in eight steps, writing each to `data/*.csv`. Walk the steps:

- Monthly volume.** Start from ~90 applications/month, multiply by the seasonal factor (Jan  $\times 2.5$ , Feb  $\times 1.6$  for renewal season, Dec  $\times 0.7$ ) and a  $\pm 10\%$  random wobble (`rnorm`). Renewal season is real: PH businesses renew permits in Jan–Feb.
- Applications.** Spread that month's count across random days; assign each a **type** (new / renewal / amendment — renewal-heavy in Jan–Feb) and a **complexity** (new→complex, else simple). Stamp a `submitted_at` timestamp during business hours and compute the RA 11032 `deadline_at` using the real working-day counter.
- Businesses.** Fewer businesses than applications (some renew every year), assigned to weighted barangays. A "Zipf-ish" weighting means a handful of businesses account for lots of applications — realistic reuse.

#### 4. Assignments (reviews). The key modelling rule:

- *Complex* (new) applications fan out to **three parallel reviews**: BPLO + CHO + BFP
- *Simple* (renewal/amendment) applications take a **BPLO-only fast path** — matching real LGU practice, and letting them realistically hit the tight 3-day deadline.

Each review's duration is a lognormal draw around the department's target mean, plus a 10% chance of a "returned → resubmitted" loop adding 2–5 days. **And here is the injected anomaly:**

```
anomaly_mult = ifelse(department_code == "CHO" &  
                      as.Date(assigned_at) >= anomaly_start, 1.8, 1.0),  
service_days = (base_days + loop_days) * anomaly_mult
```

Read: "if this is a CHO review in the last 6 weeks, multiply its duration by 1.8; otherwise leave it alone." That one line is the fault SPC later catches. It's commented in the file precisely so the paper can say it was *found*, not faked.

5. **Inspections.** Complex apps also get a sanitary (CHO) and a fire (BFP) inspection, with department-specific durations drawn from a **gamma** distribution — fire inspections average ~2.5 days vs sanitary ~1.5 (site walk-throughs take longer). This makes fire a genuine bottleneck the "+2 fire inspectors" scenario can relieve.
6. **Outcomes.** Decide each application's `status` (older ones almost all approved; very recent ones a mix of approved/in-flight/rejected) and, for approved ones, an `approved_at` = pipeline-finish + a short clerical delay.
7. **Permits.** Each approved application issues three permit types (BUSINESS, SANITARY, FSIC) with calendar-year validity.
8. **Write CSVs** and print a summary message (row counts + the anomaly window), so the console shows the fault was injected on purpose.

**How to narrate this file:** "It's a data factory. It simulates three years of permits with real-world texture — renewal seasons, uneven barangays, parallel reviews for new businesses, longer fire inspections — and it deliberately breaks CHO for the last six weeks so we have a known fault to detect. Same seed, identical dataset, every time."

## R/analytics.R – the shared calculations

Three small functions the paper names by name; each takes a table and returns a summary tibble. They're deliberately simple — the point is that they're the *shared*, *cited* definitions.

### `compute_ra11032_compliance_rate(applications)`

Of approved applications, what share beat their deadline ( `approved_at <= deadline_at` ), overall and by type. Returns count, on-time count, and rate.

### `average_processing_time(applications)`

Mean and median calendar days from submission to approval, overall and by type.

### `department_workload_stats(assignments)`

Per department: number of reviews, average turnaround, and share of total workload. (This is the fully-annotated example from Part 2.)

**The one bug worth knowing about (it shows maturity to mention).** Inside the compliance summary, the rate *must* be computed before the `on_time` column is overwritten by its own sum:

```
compliance_rate = mean(on_time), # rate FIRST (mean of TRUE/FALSE = a proportion)
on_time          = sum(on_time)  # THEN overwrite with the count
```

In R, `mean()` of a TRUE/FALSE vector is the proportion of TRUEs — a neat trick for "what fraction passed." If you summed first, `mean` would read the single summed number and be wrong. Ordering matters; this was found and fixed.

## R/spc.R – Feature 7, control charts

Four functions turn the assignments table into control charts + a flags file:

### `weekly_turnaround(assignments)`

Bucket reviews into ISO weeks by completion date, and compute each week's average turnaround per department. Drops weeks with fewer than 3 reviews (too few to trust a mean). This is the `group_by+summarise` pattern.

### `compute_control_limits(weekly, dept)`

Feed the department's **first ~24 weeks** to `qcc(type="xbar.one")` and read back the centre line, UCL, and LCL. The 24-week window is the calibration trick from Part 3 — limits set on clean history.

### `detect_processing_anomalies(weekly)`

For each department, flag a week if it's beyond the limits *or* an `ewma` drift rule trips. Returns every week tagged `in_control / out_of_control` with which rule fired.

### `.spc_chart(...)` / `run_spc(...)`

Draw the ggplot chart (blue centre line, red dashed limits, red dots for out-of-control weeks) and save a PNG per department plus `flags.csv`. The dot on the chart is the caught anomaly.

Result: CHO shows 6 consecutive red weeks in the injected window; BPLO/BFP show at most isolated one-off points (kept honestly, not hidden — a single stray point is normal noise, a 6-week run is a shift). The *run*, not the stray dot, is the signal.

## R/des.R – Feature 6, the simulation

The most conceptually dense file, but it maps cleanly onto Part 3's DES vocabulary:

### `fit_service_distributions(assignments, inspections)`

Learn the service-time distributions *from the generated history* using `fitdistrplus::fitdist(x, "lnorm")` — **excluding the anomaly weeks**, so the simulation models the *normal* office, not the broken fortnight. Returns each stage's lognormal parameters.

### `build_pipeline(env, resources, params)`

Add the five resources (BPLO, CHO, BFP, sanitary, fire) at given capacities, and define two trajectories: the **complex** route (payment delay → clone into 3 parallel reviews → synchronize → clone into 2 parallel inspections → synchronize → issuance) and the **simple** route (payment → BPLO review → issuance).

### `simulate_staffing_scenario(...)`

Run `DES_REPS` (30) replications, each with `set.seed(SEED + r)`, generating arrivals via `rexp`, then compute on-time compliance and per-resource wait/queue/utilisation. Note the `7/5` working-day conversion and the `eligible` filter (only judge applications that entered early enough to have *had a chance* to finish before the horizon — avoids penalising late arrivals unfairly).

### `run_des(...)`

Run three scenarios — `baseline`, `plus2_bfp_inspectors`, `plus1_cho_reviewer` — write `scenarios.csv`, and draw the comparison bar chart (dashed line = baseline).

**How to narrate it:** "We fit realistic service times from our own clean history, build a simmer model of the exact permit flow — parallel reviews and inspections included — then run it 30 times under current staffing and under two 'add staff' scenarios. Both additions raise on-time compliance, and the utilisation numbers show *why*: they relieve the busiest resource."

## plumber.R + run\_api.R – the API

A tiny web service. The special `##` comments are `plumber` annotations that turn a plain R function into an HTTP endpoint. Three endpoints, all serving *precomputed* results as JSON:

```
## @get /health          → {"status":"ok", ...}
## @get /spc/flags       → the weekly control-chart flags
## @get /des/scenarios   → the staffing-scenario results
```

`run_api.R` starts it on port 8787. The architecture point: R does the heavy analytics *offline*, and the main BizTrack app (Laravel) would just fetch the finished results over HTTP — it never runs R live. If asked "how does R talk to the rest of the system?" — this file is the answer.

## R/db.R – the future database readers (stubs)

Deliberately inert. Five functions named `read*_pg` hold the real SQL to read from the eventual PostgreSQL database, but every one calls `.require_postgres()` first, which **stops** unless `MODE == "postgres"`. They exist so the integration path is visible in code review, and so that *when* the real DB is wired up, the analytics/SPC/DES code doesn't change — the readers just return tibbles with the same column names the generator emits. It's a documented seam, not dead code.

## run\_all.R – the one command

The conductor. It sources config + every module, then runs the four stages with progress messages, and finally writes `outputs/summary.md` (a standalone report). Reading its `[1/4] ... [4/4]` prints top to bottom is reading the whole pipeline. Two small craftsmanship notes you can point at: the `gcat()` helper exists because `glue()` strips trailing newlines by default (so status lines would run together), and the numbers printed at the end (baseline vs each scenario, with the delta) are the exact figures you'd read aloud in a defense.

## app.R – the Shiny dashboard

**Shiny** is R's web-app framework: you write R, it serves an interactive dashboard — no JavaScript. It's the conventional way to present an R analysis, which is why it's here for the progress report. Every Shiny app is two halves:

## ui

the layout — what the page looks like. Here, `page_navbar` with four tabs (Overview, SPC, DES, Analytics tables), value boxes for the headline numbers, and cards holding charts and tables. Styled in civic blue ( `#0025cc` ) to match BizTrack.

## server

the logic — what fills those slots. `render*` functions compute outputs: `renderPlot` draws the charts, `renderDT` draws interactive tables. The SPC tab is **reactive**: pick a department in the dropdown and the chart *re-renders* for it, because `renderPlot({ .spc_chart(weekly, flags, input$dept) })` re-runs whenever `input$dept` changes.

Crucially, the app **reads the precomputed CSVs** (it re-uses `config.R`, `analytics.R`, `spc.R`) — it does *not* re-run the heavy simulation. So you must run `Rscript run_all.R` first (to make `data/` and `outputs/`), then `Rscript run_app.R` to serve it at `http://localhost:8788`. The app even checks the files exist and gives a clear error if you skipped step one.

## install.R – the installer

Run-once setup: it installs any of the ~16 packages that are missing (re-running is a harmless no-op). The interesting part is a macOS **preflight**: some Homebrew R builds hard-code a C-compiler flag ( `-std=gnu23` ) that Apple's older clang rejects, which would make every package fail to compile. `ensure_c_std()` detects exactly that situation and writes a one-line override to `~/R/Makevars` to fix it. It's the reason "it just installs" on your Mac; you hit that wall during the build and made the installer self-heal.

## Part 5 – Explain it like you made it

---

The panel won't quiz you on R syntax. They'll poke your *choices*. Here are the ones worth rehearsing, phrased as answers.

### "Why fake data? Isn't that cheating?"

There's no production permit database yet, and we can't publish real citizens' records anyway. So we built a generator with realistic structure — seasonality, parallel departmental reviews, lognormal service times — and, importantly, we seed it so it's *reproducible*. The paper's Feature 6 explicitly reads two ways: "simulate data *into* the system" and "simulate the system to get answers." The generator is the first; the DES is the second.

### "How do you know your SPC actually works?"

Because we ran a controlled experiment. We *injected* a known fault — CHO reviews  $1.8\times$  slower for the last six weeks — and then showed the control chart flags exactly those six weeks as out-of-control, while the untouched departments stay in control. Detection of a planted fault is the proof. And we calibrated the control limits on the *clean* first 24 weeks so the slowdown couldn't hide itself by inflating its own limits.

### "Why lognormal for durations?"

Processing times can't be negative, are usually short, and occasionally drag on — that's the textbook lognormal shape, standard for service times. We even back-solve the lognormal parameter so we can target an exact average per department.

### "Your DES compliance is ~66%. Isn't that low?"

That's the baseline under a deliberately stressed, steady caseload — the whole point is that it's a lever we can move. Adding fire inspectors or a CHO reviewer both raise it, and the utilisation numbers show why: they relieve the busiest resource. We tuned the arrival rate to a plausible band (65–90%) rather than raw seasonal peaks, and we documented that arrivals-seasonality is future work — we didn't fudge the number, we calibrated it and said so.

### "Why the 7/5 multiplier in the deadline?"

RA 11032 deadlines are in *working* days; our simulation clock is in *calendar* days. A 5-day work week spans 7 calendar days, so we scale the working-day deadline by 7/5 to compare like with like. Without it the compliance number would be silently wrong.

### "How would this connect to the real BizTrack app?"

R runs the analytics offline and writes results; `pLumber.R` exposes them as a JSON API; the Laravel app fetches the finished numbers over HTTP. R never runs live inside a web request. And `db.R` already holds the SQL to swap the synthetic CSVs for the real database when it's ready — nothing downstream changes because the column names match.

## "Why is everything reproducible from one number?"

`SEED <- 1103` in `config.R`. Every random step derives from it, so re-running `Rscript run_all.R` reproduces identical data, charts, and scenario numbers — on any machine. That's a requirement for a defensible paper.

## One-page cheat sheet

| If you see...                                                                              | Say...                                                              |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>&lt;-</code>                                                                         | "assignment — x gets a value"                                       |
| <code> &gt;</code>                                                                         | "pipe — 'then'; feeds the left into the next function"              |
| <code>c(...)</code>                                                                        | "combine into a vector"                                             |
| <code>filter</code> / <code>mutate</code> / <code>group_by</code> / <code>summarise</code> | "keep rows / add column / bucket / collapse each bucket to one row" |
| <code>left_join</code>                                                                     | "glue two tables on a shared key"                                   |
| <code>set.seed(1103)</code>                                                                | "make the randomness reproducible"                                  |
| <code>rlnorm</code> / <code>rgamma</code> / <code>runif</code> / <code>rexp</code>         | "draw random durations/delays of a realistic shape"                 |
| <code>qcc(type="xbar.one")</code>                                                          | "build the control chart; flag out-of-control weeks"                |
| calibrate on first 24 weeks                                                                | "so the injected slowdown can't hide itself"                        |
| <code>simmer</code> / <code>seize</code> / <code>timeout</code> / <code>release</code>     | "the simulation — grab staff, spend service time, release"          |
| <code>clone</code> + <code>synchronize</code>                                              | "model the 3 parallel departmental reviews"                         |
| utilisation $\approx$ 0.9                                                                  | "that resource is a bottleneck"                                     |
| <code>anomaly_mult ... 1.8</code>                                                          | "the fault we injected for SPC to catch"                            |
| <code>plumber</code> <code>#*</code> <code>@get</code>                                     | "expose the results as a JSON API for Laravel"                      |

**The 3-sentence summary to memorise.** "We generate 36 months of reproducible synthetic permit history — including a deliberately injected slowdown in one department. Feature 7 (SPC control charts) catches that slowdown statistically, proving the detection works. Feature 6 (a simmer discrete-event simulation) models the office and shows, with numbers, that adding staff improves RA 11032 on-time compliance — so staffing decisions can be tested before they're made."